

变量作用域

复习

- | | | |
|-----|-------------|----|
| 1. | 函数概述 | 了解 |
| 2. | 函数类型 | 了解 |
| 3. | 函数定义与声明 | 重点 |
| 4. | 函数参数 | 重点 |
| 5. | 函数可变参数 | 熟悉 |
| 6. | 函数默认参数 | 熟悉 |
| 7. | 函数返回值 | 重点 |
| 8. | 函数嵌套调用 | 熟悉 |
| 9. | 函数递归 | 重点 |
| 10. | 内联函数 | 熟悉 |
| 11. | 内联函数与普通函数区别 | 了解 |
| 12. | 函数重载 | 重点 |

目录

- | | |
|---------------|----|
| 1. 概念 | 了解 |
| 2. typedef 声明 | 了解 |
| 3. 局部变量 | 重点 |
| 4. 全局变量 | 重点 |
| 5. 静态变量 | 熟悉 |

概念

C++ 中的一个 变量 或 常量 在程序中都有一定的作用范围，称之为作用域。

变量定义

作用：给一段指定的内存空间起名，方便操作这段内存,相当于被某块内存起了个别名。

语法：数据类型 变量名 = 值;

概念

变量类型

基本变量类型有int , float , char , double等等。

变量类型不同占有内存的字节数不同。

变量类型	字节 (位)
int	4
unsigned int	4
signed in	4
char	1
unsigned char	1
signed char	1
short int	2
long int	8
float	4
double	8
long double	16

typedef 声明

typedef是在计算机编程语言中用来为复杂的声明定义简单的别名，它与宏定义有些差异。它本身是一种存储类的关键字，与auto、extern、mutable、static、register等关键字不能出现在同一个表达式中

- 1.typedef 可以声明各种类型名，但不能用来定义变量。
- 2.用typedef只是对已经存在的类型增加一个类型名，不能创造新的类型。
- 3.当在不同源文件中用到同一类型数据（尤其是像数组、指针、结构体、共用体等类型数据）时，常用 typedef 声明一些数据类型，以提高编程效率。
- 4.typedef 有利于程序的通用与移植。有时程序会依赖于硬件特性，用 typedef 便于移植。

typedef 语法

typedef 可以用来为现有的类型来命名一个新的名字
语法

```
typedef int abc ;
```

有了上述代码后，程序中使用abc声明整型变量

```
abc num = 1 ;
```

说明创建了一个int类型的变量并进行赋值1。

局部变量

在函数内部声明/定义的变量叫局部变量，局部变量的作用域仅限于函数内部。同时，在函数内部定义的变量，函数的参数和返回值，if 和 for 结构内部使用的变量等都是局部变量。

当存在全局变量和局部变量同名的时候，在函数内，局部变量会覆盖全局变量。

使用 {} 可以限制变量的作用域

局部变量 案例

```
#include <iostream>
using namespace std;
int main()
{
    cout << "测试局部变量" << endl;
    {
        int num = 100 ; //局部
        cout << "num = " << num << endl;
    }
    //cout << "num = " << num << endl;
}
```

全局变量

在函数外部声明/定义的变量叫全局变量，C++ 全局变量作用域在整个文件都有效。

全局变量 案例

```
#include <iostream>
using namespace std;
int num = 100 ; //全局
void test ( ) {
    cout << "test func print num = " << num << endl;
}
int main()
{
    cout << "测试全局变量" << endl;
    cout << "num = " << num << endl;
    cout << "-----" << endl;
    test();
}
```

静态变量

静态存储区的变量是在程序刚开始的时候就会完成初始化，也是整个程序生命周期的唯一一次初始化。

静态存储区中一般存储两种变量：static 变量和全局变量。

static变量在函数内定义局部变量的时候，作用域只在其函数内，但是生存周期为整个源程序。即只可以在定义该变量的函数内进行使用，但是退出该函数后，该变量一直存在，但不可以进行使用。

静态变量 案例

```
int num = 1; //全局
```

```
int func(){
```

```
    static int num = 5 ; //首次进入函数时，num初始化，
```

```
    return num--; //后面在进入函数时，num不会被初始化
```

```
}
```

```
int mian(){
```

```
    for(;num <=5; num++){
```

```
        cout << num <<"---" << func() << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

C++存储类

存储类用于定义C++程序中变量和函数的生命周期和可见性。

寿命是指变量保持活动的时间段，可见性是指可访问变量的程序的模块。

有五种类型的存储类，可以在C++程序中使用

1. 自动(Automatic)
2. 寄存器(Register)
3. 静态(Static)
4. 外部(External)
5. 可变(Mutable)

C++存储类

存储类	关键字	生命周期	可见性	初始值
自动	auto	函数块	局部	垃圾
寄存器	register	函数块	局部	垃圾
可变	mutable	类	局部	垃圾
静态	static	整个程序	全局	零
外部	extern	整个程序	局部	零

auto 自动存储类

auto是所有局部变量的默认存储类。

auto关键字自动应用于所有局部变量。

```
{  
    auto int v1;  
    float v2 = 99.1;  
}
```

上面的例子定义了两个具有相同存储类的变量，**auto只能在函数中使用。**

static 静态存储类

静态变量只初始化一次，直到程序结束。

它保留可在多个函数之间调用的值。

静态变量由编译器提供的一个默认值：**0**。

```
void func() {  
    static int v1=0; //static variable  
    int v2=0; //local variable  
    v1++;  
    v2++;  
    cout<<"v1=" << v1<<"    v2=" <<v2<<endl;  
}
```



多调用几次func函数

register 注册存储类

寄存器变量在寄存器中分配存储器而不是RAM。其大小与寄存器大小相同。它比其他变量具有更快的访问速度。

建议仅使用寄存器变量进行快速访问!

ex:在计数器中。

```
register int count=0;
```

注意：不能得到寄存器变量的地址。

extern 外部存储类

extern变量对所有程序都可见。

如果两个或多个文件共享相同的变量或函数，则使用它。

```
extern int v1 = 0;
```

课堂练习

利用全局变量统计函数被调用次数

课堂练习 代码

```
int count = 0 ;  
void func(){  
    count ++;  
}  
  
int main(){  
    for(int i=0; i<100; i++){  
        func();  
    }  
    cout << "func函数被调用次数：" << count << endl;  
    return 0;  
}
```

知识点总结

- | | |
|---------------|----|
| 1. 概念 | 了解 |
| 2. typedef 声明 | 了解 |
| 3. 局部变量 | 重点 |
| 4. 全局变量 | 重点 |
| 5. 静态变量 | 熟悉 |
| 6. C++存储类 | 熟悉 |

作业

1. 简述全局变量和局部变量的作用域及生命周期

下节课预告

多文件编程